

09 - Material - Lista linear

May 25, 2026

Aula: Listas Lineares em C

Vetores, lista encadeada simples, lista circular, lista duplamente encadeada e lista duplamente encadeada circular

Este material apresenta os principais conceitos de listas lineares e suas implementações em linguagem C.

A aula foi organizada para alunos que estão iniciando o estudo de Estrutura de Dados, com explicações conceituais, representações visuais em texto, exemplos de código e exercícios.

Objetivos da aula

Ao final desta aula, o estudante deverá ser capaz de:

- compreender o conceito de lista linear;
- diferenciar vetor e lista encadeada;
- implementar operações básicas em vetores usando C;
- entender o funcionamento de ponteiros em listas encadeadas;
- implementar lista encadeada simples;
- compreender lista circular, lista duplamente encadeada e lista duplamente encadeada circular;
- comparar as principais características dessas estruturas;
- escolher a estrutura mais adequada para diferentes situações.

Como usar os códigos C no VSCode/Jupyter

Este notebook foi ajustado para permitir a compilação dos exemplos em C diretamente dentro do VSCode com a extensão do Jupyter.

Antes dos exemplos, execute a célula abaixo para criar a pasta onde os arquivos .c e os executáveis serão gravados.

Depois, para cada exemplo:

1. Execute a célula `%%writefile` para gravar o código em um arquivo .c.
2. Execute a célula seguinte para compilar com gcc e executar o programa.

No macOS e no Linux, o comando usado é:

```
gcc arquivo.c -o executavel && ./executavel
```

No Windows com MinGW, adapte a execução para .exe quando necessário.

```
[2]: import os
os.makedirs("codigos_c", exist_ok=True)
print("Pasta codigos_c criada ou já existente.")
```

Pasta codigos_c criada ou já existente.

1 O que é uma lista linear?

Uma **lista linear** é uma estrutura de dados em que os elementos são organizados em uma sequência.

Cada elemento possui uma posição lógica dentro da lista.

Exemplo:

[10] [20] [30] [40] [50]

Essa organização é chamada de linear porque existe uma relação de ordem entre os elementos:

10 vem antes de 20
20 vem antes de 30
30 vem antes de 40
40 vem antes de 50

Em uma lista linear, normalmente realizamos operações como:

- inserir elementos;
- remover elementos;
- buscar elementos;
- percorrer todos os elementos;
- alterar valores;
- verificar se a lista está vazia;
- verificar o tamanho da lista.

2 Formas de implementar uma lista linear

Uma lista linear pode ser implementada de várias formas. Nesta aula, estudaremos as seguintes:

Lista Linear

```
|
|-- Vetor
|
|-- Lista Encadeada Simples
|
|-- Lista Encadeada Circular
|
|-- Lista Duplamente Encadeada
|
|-- Lista Duplamente Encadeada Circular
```

A principal diferença entre essas estruturas está na forma como os dados são armazenados e ligados na memória.

3 Lista linear usando vetor

3.1 Conceito

Um **vetor** é uma sequência de posições consecutivas na memória.

Em C, podemos declarar um vetor de inteiros assim:

```
int numeros[5];
```

Esse vetor possui 5 posições:

Índice:	0	1	2	3	4
Valor:	?	?	?	?	?

Depois de preencher o vetor:

```
int numeros[5] = {10, 20, 30, 40, 50};
```

Temos:

Índice:	0	1	2	3	4
Valor:	10	20	30	40	50

3.2 Observação importante

Em C, os índices começam em 0.

Portanto, em um vetor com 5 elementos, os índices válidos são:

0, 1, 2, 3, 4

O índice 5 não pertence ao vetor.

4 Características do vetor

O vetor possui algumas características importantes:

1. Os elementos ficam em posições consecutivas da memória.
2. O acesso a qualquer posição é direto.
3. O tamanho normalmente é definido antes do uso.
4. Inserções e remoções no meio exigem deslocamento de elementos.
5. É uma estrutura simples e eficiente quando o tamanho é conhecido.

Exemplo de acesso direto:

```
printf("%d", numeros[2]);
```

Se `numeros[2]` contém 30, a saída será:

30

5 Criando e exibindo uma lista usando vetor

Neste exemplo, usamos um vetor com capacidade para 10 elementos, mas inicialmente armazenamos apenas 3 valores.

A variável `quantidade` controla quantos elementos estão sendo usados.

```
[3]: %%writefile codigos_c/exemplo_01.c

#include <stdio.h>

#define TAM 10

int main() {
    int lista[TAM];
    int quantidade = 0;

    lista[0] = 10;
    lista[1] = 20;
    lista[2] = 30;

    quantidade = 3;

    printf("Elementos da lista:\n");

    for (int i = 0; i < quantidade; i++) {
        printf("%d ", lista[i]);
    }

    return 0;
}
```

Writing codigos_c/exemplo_01.c

```
[4]: !gcc codigos_c/exemplo_01.c -o codigos_c/exemplo_01 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_01
```

```
Elementos da lista:
10 20 30
```

5.1 Explicação

`TAM` representa a capacidade máxima do vetor.

`quantidade` representa quantos elementos foram inseridos.

O vetor pode armazenar até 10 elementos, mas somente os 3 primeiros estão sendo utilizados.

6 Inserção no final do vetor

Inserir no final é simples quando ainda existe espaço disponível.

Situação inicial:

10 20 30

Inserindo 40 no final:

10 20 30 40

```
[5]: %%writefile codigos_c/exemplo_02.c

#include <stdio.h>

#define TAM 10

int main() {
    int lista[TAM] = {10, 20, 30};
    int quantidade = 3;

    if (quantidade < TAM) {
        lista[quantidade] = 40;
        quantidade++;
    } else {
        printf("Lista cheia.\n");
    }

    printf("Lista após inserção:\n");

    for (int i = 0; i < quantidade; i++) {
        printf("%d ", lista[i]);
    }

    return 0;
}
```

Writing codigos_c/exemplo_02.c

```
[6]: !gcc codigos_c/exemplo_02.c -o codigos_c/exemplo_02 -Wall -Wextra -std=c11 && ./
↳codigos_c/exemplo_02
```

Lista após inserção:

10 20 30 40

6.1 Análise

A posição `lista[quantidade]` representa a próxima posição livre.

Antes da inserção:

`quantidade = 3`

`próxima posição livre = lista[3]`

Depois da inserção:

```
lista[3] = 40
quantidade = 4
```

7 Inserção em uma posição específica do vetor

Para inserir um elemento no meio do vetor, precisamos deslocar os elementos para a direita.

Exemplo: inserir 25 na posição 2.

Antes:

Índice:	0	1	2	3
Valor:	10	20	30	40

Depois:

Índice:	0	1	2	3	4
Valor:	10	20	25	30	40

O valor 30 e o valor 40 foram deslocados.

```
[7]: %%writefile codigos_c/exemplo_03.c

#include <stdio.h>

#define TAM 10

int main() {
    int lista[TAM] = {10, 20, 30, 40};
    int quantidade = 4;
    int valor = 25;
    int posicao = 2;

    if (quantidade < TAM && posicao >= 0 && posicao <= quantidade) {

        for (int i = quantidade; i > posicao; i--) {
            lista[i] = lista[i - 1];
        }

        lista[posicao] = valor;
        quantidade++;

    } else {
        printf("Não foi possível inserir.\n");
    }

    printf("Lista após inserção:\n");

    for (int i = 0; i < quantidade; i++) {
        printf("%d ", lista[i]);
    }
}
```

```

    }

    return 0;
}

```

Writing `codigos_c/exemplo_03.c`

```

[8]: !gcc codigos_c/exemplo_03.c -o codigos_c/exemplo_03 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_03

```

Lista após inserção:

10 20 25 30 40

7.1 Ideia do deslocamento

O deslocamento precisa começar do final para o início.

Se fizéssemos o deslocamento do início para o final, poderíamos sobrescrever valores importantes.

Sequência do deslocamento:

40 vai para a próxima posição

30 vai para a próxima posição

25 entra na posição livre

8 Remoção em vetor

Para remover um elemento do vetor, os elementos seguintes precisam ser deslocados para a esquerda.

Exemplo: remover o elemento da posição 1.

Antes:

Índice:	0	1	2	3
Valor:	10	20	30	40

Depois:

Índice:	0	1	2
Valor:	10	30	40

```

[ ]: %%writefile codigos_c/exemplo_04.c

#include <stdio.h>

#define TAM 10

int main() {
    int lista[TAM] = {10, 20, 30, 40};
    int quantidade = 4;
    int posicao = 1;

```

```

    if (posicao >= 0 && posicao < quantidade) {

        for (int i = posicao; i < quantidade - 1; i++) {
            lista[i] = lista[i + 1];
        }

        quantidade--;

    } else {
        printf("Posição inválida.\n");
    }

    printf("Lista após remoção:\n");

    for (int i = 0; i < quantidade; i++) {
        printf("%d ", lista[i]);
    }

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_04.c -o codigos_c/exemplo_04 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_04

```

9 Busca em vetor

A busca consiste em procurar um determinado valor dentro da lista.

Neste exemplo, vamos procurar o valor 30.

```

[ ]: %%writefile codigos_c/exemplo_05.c

#include <stdio.h>

#define TAM 10

int main() {
    int lista[TAM] = {10, 20, 30, 40, 50};
    int quantidade = 5;
    int valorBuscado = 30;
    int encontrado = 0;

    for (int i = 0; i < quantidade; i++) {
        if (lista[i] == valorBuscado) {
            printf("Valor encontrado na posição %d.\n", i);
            encontrado = 1;
            break;
        }
    }
}

```



```

    }
}

if (!encontrado) {
    printf("Valor não encontrado.\n");
}

return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_05.c -o codigos_c/exemplo_05 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_05

```

10 Limitações do vetor

O vetor é uma estrutura muito importante, mas possui limitações.

10.1 Pontos positivos

- Implementação simples.
- Acesso rápido por índice.
- Boa organização na memória.
- Excelente para listas de tamanho conhecido.

10.2 Pontos negativos

- Tamanho geralmente fixo.
- Pode desperdiçar memória.
- Inserção no início ou no meio exige deslocamento.
- Remoção no início ou no meio exige deslocamento.

10.3 Exemplo prático

Se uma turma sempre possui no máximo 40 alunos, um vetor pode ser adequado.

Mas se a quantidade de elementos cresce e diminui constantemente, uma lista encadeada pode ser mais interessante.

11 Lista encadeada simples

11.1 Conceito

Uma **lista encadeada simples** é formada por nós.

Cada nó possui duas partes:

1. O valor armazenado
2. O endereço do próximo nó

Representação:

[10 | próximo] -> [20 | próximo] -> [30 | NULL]

O último nó aponta para NULL, indicando o fim da lista.

11.2 Diferença em relação ao vetor

No vetor, os elementos ficam lado a lado na memória.

Na lista encadeada, os elementos podem estar espalhados na memória. O que mantém a sequência é o ponteiro para o próximo nó.

12 Ponteiros e alocação dinâmica

Para trabalhar com lista encadeada em C, precisamos usar ponteiros e alocação dinâmica de memória.

12.1 Ponteiro

Um ponteiro armazena o endereço de uma variável ou estrutura.

Exemplo:

```
int x = 10;
int *p = &x;
```

12.2 Alocação dinâmica

A função `malloc` reserva memória durante a execução do programa.

Exemplo:

```
No *novo = malloc(sizeof(No));
```

A função `free` libera a memória reservada.

Exemplo:

```
free(novo);
```

12.3 Importante

Sempre que usamos `malloc`, em algum momento devemos usar `free` para evitar desperdício de memória.

13 Estrutura de um nó

Em C, podemos representar um nó assim:

```
[ ]: %%writefile codigos_c/exemplo_06.c

#include <stdio.h>

typedef struct No {
```

```

    int valor;
    struct No *proximo;
} No;

int main() {
    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_06.c -o codigos_c/exemplo_06 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_06

```

13.1 Explicação

`int valor;`

Armazena o dado do nó.

`struct No *proximo;`

Armazena o endereço do próximo nó da lista.

`typedef`

Permite usar o nome `No` diretamente, sem precisar escrever `struct No` toda vez.

14 Criando manualmente uma lista encadeada simples

Neste exemplo, criaremos três nós manualmente.

```

[ ]: %%writefile codigos_c/exemplo_07.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *proximo;
} No;

int main() {
    No *primeiro = malloc(sizeof(No));
    No *segundo = malloc(sizeof(No));
    No *terceiro = malloc(sizeof(No));

    primeiro->valor = 10;
    primeiro->proximo = segundo;

    segundo->valor = 20;
    segundo->proximo = terceiro;
}

```

```

    terceiro->valor = 30;
    terceiro->proximo = NULL;

    No *atual = primeiro;

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");

    free(primeiro);
    free(segundo);
    free(terceiro);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_07.c -o codigos_c/exemplo_07 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_07

```

14.1 Representação da lista

```

primeiro
  |
  v
[10 | *] -> [20 | *] -> [30 | NULL]

```

15 Inserção no início da lista encadeada simples

Inserir no início é uma operação eficiente.

Situação inicial:

20 -> 30 -> NULL

Inserindo 10 no início:

10 -> 20 -> 30 -> NULL

O novo nó passa a apontar para o antigo início.

```

[ ]: %%writefile codigos_c/exemplo_08.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;

```

```

    struct No *proximo;
} No;

No* inserirInicio(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;
    novo->proximo = inicio;

    return novo;
}

void exibirLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

void liberarLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
}

int main() {
    No *lista = NULL;

    lista = inserirInicio(lista, 30);
    lista = inserirInicio(lista, 20);
    lista = inserirInicio(lista, 10);

    exibirLista(lista);
}

```

```

    liberarLista(lista);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_08.c -o codigos_c/exemplo_08 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_08

```

16 Inserção no final da lista encadeada simples

Para inserir no final, precisamos percorrer a lista até encontrar o último nó.

O último nó é aquele cujo `proximo` é `NULL`.

10 -> 20 -> 30 -> NULL

Inserindo 40:

10 -> 20 -> 30 -> 40 -> NULL

```

[ ]: %%writefile codigos_c/exemplo_09.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *proximo;
} No;

No* inserirFinal(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;
    novo->proximo = NULL;

    if (inicio == NULL) {
        return novo;
    }

    No *atual = inicio;

    while (atual->proximo != NULL) {
        atual = atual->proximo;
    }
}

```

```

    }

    atual->proximo = novo;

    return inicio;
}

void exibirLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

void liberarLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
}

int main() {
    No *lista = NULL;

    lista = inserirFinal(lista, 10);
    lista = inserirFinal(lista, 20);
    lista = inserirFinal(lista, 30);

    exibirLista(lista);

    liberarLista(lista);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_09.c -o codigos_c/exemplo_09 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_09

```

17 Busca em lista encadeada simples

A busca percorre a lista nó por nó até encontrar o valor desejado ou chegar ao fim.

```
[ ]: %%writefile codigos_c/exemplo_10.c

#include <stdio.h>
#include <stdlib.h>
typedef struct No {
    int valor;
    struct No *proximo;
} No;

void buscarValor(No *inicio, int valor) {
    No *atual = inicio;
    int posicao = 0;

    while (atual != NULL) {
        if (atual->valor == valor) {
            printf("Valor %d encontrado na posição %d.\n", valor, posicao);
            return;
        }

        atual = atual->proximo;
        posicao++;
    }

    printf("Valor %d não encontrado.\n", valor);
}

int main() {
    return 0;
}
```

```
[ ]: !gcc codigos_c/exemplo_10.c -o codigos_c/exemplo_10 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_10
```

18 Remoção em lista encadeada simples

Para remover um nó, precisamos ajustar os ponteiros.

Exemplo:

10 -> 20 -> 30 -> NULL

Removendo 20:

10 -> 30 -> NULL

O nó 10 deixa de apontar para 20 e passa a apontar para 30.


```
[ ]: %%writefile codigos_c/exemplo_11.c
```

```
#include <stdio.h>
#include <stdlib.h>
typedef struct No {
    int valor;
    struct No *anterior;
    struct No *proximo;
} No;

No* removerValor(No *inicio, int valor) {
    No *atual = inicio;
    No *anterior = NULL;

    while (atual != NULL && atual->valor != valor) {
        anterior = atual;
        atual = atual->proximo;
    }

    if (atual == NULL) {
        printf("Valor %d não encontrado.\n", valor);
        return inicio;
    }

    if (anterior == NULL) {
        inicio = atual->proximo;
    } else {
        anterior->proximo = atual->proximo;
    }

    free(atual);

    return inicio;
}

int main() {
    return 0;
}
```

```
[ ]: !gcc codigos_c/exemplo_11.c -o codigos_c/exemplo_11 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_11
```

19 Exemplo completo: lista encadeada simples

O código a seguir reúne inserção, exibição, busca, remoção e liberação de memória.

```

[ ]: %%writefile codigos_c/exemplo_12.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *proximo;
} No;

No* inserirFinal(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;
    novo->proximo = NULL;

    if (inicio == NULL) {
        return novo;
    }

    No *atual = inicio;

    while (atual->proximo != NULL) {
        atual = atual->proximo;
    }

    atual->proximo = novo;

    return inicio;
}

No* removerValor(No *inicio, int valor) {
    No *atual = inicio;
    No *anterior = NULL;

    while (atual != NULL && atual->valor != valor) {
        anterior = atual;
        atual = atual->proximo;
    }

    if (atual == NULL) {
        printf("Valor %d não encontrado.\n", valor);
    }
}

```

```

        return inicio;
    }

    if (anterior == NULL) {
        inicio = atual->proximo;
    } else {
        anterior->proximo = atual->proximo;
    }

    free(atual);

    return inicio;
}

void buscarValor(No *inicio, int valor) {
    No *atual = inicio;
    int posicao = 0;

    while (atual != NULL) {
        if (atual->valor == valor) {
            printf("Valor %d encontrado na posição %d.\n", valor, posicao);
            return;
        }

        atual = atual->proximo;
        posicao++;
    }

    printf("Valor %d não encontrado.\n", valor);
}

void exibirLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

void liberarLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        No *proximo = atual->proximo;

```

```

        free(atual);
        atual = proximo;
    }
}

int main() {
    No *lista = NULL;

    lista = inserirFinal(lista, 10);
    lista = inserirFinal(lista, 20);
    lista = inserirFinal(lista, 30);
    lista = inserirFinal(lista, 40);

    printf("Lista inicial:\n");
    exibirLista(lista);

    buscarValor(lista, 30);

    lista = removerValor(lista, 20);

    printf("Lista após remover 20:\n");
    exibirLista(lista);

    liberarLista(lista);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_12.c -o codigos_c/exemplo_12 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_12

```

20 Lista encadeada circular

20.1 Conceito

Na lista encadeada circular, o último nó aponta novamente para o primeiro nó.

Em uma lista simples comum:

10 -> 20 -> 30 -> NULL

Em uma lista circular:

```

10 -> 20 -> 30
^         |
|         |
|_____|

```

A lista circular não termina em NULL.

O fim lógico da lista é identificado quando voltamos ao nó inicial.

20.2 Aplicações

Listas circulares são úteis quando os elementos precisam ser percorridos repetidamente.

Exemplos:

- escalonamento de processos em sistemas operacionais;
- alternância de jogadores em um jogo;
- playlists que retornam à primeira música;
- carrossel de imagens;
- sistemas com funcionamento cíclico.

21 Inserção e exibição em lista circular simples

A estrutura do nó é a mesma da lista encadeada simples.

A diferença está no fato de que o último nó aponta para o início.

```
[ ]: %%writefile codigos_c/exemplo_13.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *proximo;
} No;

No* inserirCircular(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;

    if (inicio == NULL) {
        novo->proximo = novo;
        return novo;
    }

    No *atual = inicio;

    while (atual->proximo != inicio) {
        atual = atual->proximo;
    }
```

```

    atual->proximo = novo;
    novo->proximo = inicio;

    return inicio;
}

void exibirCircular(No *inicio) {
    if (inicio == NULL) {
        printf("Lista vazia.\n");
        return;
    }

    No *atual = inicio;

    do {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    } while (atual != inicio);

    printf("(volta ao início)\n");
}

void liberarCircular(No *inicio) {
    if (inicio == NULL) {
        return;
    }

    No *atual = inicio->proximo;

    while (atual != inicio) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }

    free(inicio);
}

int main() {
    No *lista = NULL;

    lista = inserirCircular(lista, 10);
    lista = inserirCircular(lista, 20);
    lista = inserirCircular(lista, 30);

    exibirCircular(lista);
}

```

```
    liberarCircular(lista);  
  
    return 0;  
}
```

```
[ ]: !gcc codigos_c/exemplo_13.c -o codigos_c/exemplo_13 -Wall -Wextra -std=c11 && ./  
↪codigos_c/exemplo_13
```

21.1 Atenção ao percorrer uma lista circular

Em uma lista encadeada simples, paramos quando encontramos NULL.

Em uma lista circular, não existe NULL no final.

Por isso, usamos do `while` e paramos quando voltamos ao nó inicial.

```
começa no início  
visita os nós  
para quando atual voltar a ser igual ao início
```

22 Lista duplamente encadeada

22.1 Conceito

Na lista duplamente encadeada, cada nó aponta para o próximo nó e também para o nó anterior.

Representação:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Cada nó possui:

```
valor  
ponteiro para o anterior  
ponteiro para o próximo
```

22.2 Vantagem principal

A lista duplamente encadeada permite percorrer a lista nos dois sentidos:

```
ida:    10 -> 20 -> 30  
volta:  30 -> 20 -> 10
```

22.3 Aplicações

- botão voltar e avançar de navegadores;
- histórico de navegação;
- editores com desfazer e refazer;
- playlists com música anterior e próxima;
- estruturas que exigem navegação nos dois sentidos.

23 Estrutura do nó da lista duplamente encadeada

A estrutura possui dois ponteiros:

anterior

próximo

```
[ ]: %%writefile codigos_c/exemplo_14.c
```

```
#include <stdio.h>

typedef struct No {
    int valor;
    struct No *anterior;
    struct No *proximo;
} No;

int main() {
    return 0;
}
```

```
[ ]: !gcc codigos_c/exemplo_14.c -o codigos_c/exemplo_14 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_14
```

24 Inserção no início da lista duplamente encadeada

```
[ ]: %%writefile codigos_c/exemplo_15.c
```

```
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *anterior;
    struct No *proximo;
} No;

No* inserirInicio(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;
    novo->anterior = NULL;
```



```

    novo->proximo = inicio;

    if (inicio != NULL) {
        inicio->anterior = novo;
    }

    return novo;
}

void exibirDireto(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        printf("%d <-> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

void liberarLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
}

int main() {
    No *lista = NULL;

    lista = inserirInicio(lista, 30);
    lista = inserirInicio(lista, 20);
    lista = inserirInicio(lista, 10);

    exibirDireto(lista);

    liberarLista(lista);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_15.c -o codigos_c/exemplo_15 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_15

```

25 Exibição reversa em lista duplamente encadeada

Para exibir em ordem reversa, primeiro precisamos chegar ao último nó.

Depois usamos o ponteiro anterior.

```
[ ]: %%writefile codigos_c/exemplo_16.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *anterior;
    struct No *proximo;
} No;

No* inserirFinal(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;
    novo->proximo = NULL;

    if (inicio == NULL) {
        novo->anterior = NULL;
        return novo;
    }

    No *atual = inicio;

    while (atual->proximo != NULL) {
        atual = atual->proximo;
    }

    atual->proximo = novo;
    novo->anterior = atual;

    return inicio;
}

void exibirDireto(No *inicio) {
    No *atual = inicio;
```

```

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

void exibirReverso(No *inicio) {
    if (inicio == NULL) {
        printf("Lista vazia.\n");
        return;
    }

    No *atual = inicio;

    while (atual->proximo != NULL) {
        atual = atual->proximo;
    }

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->anterior;
    }

    printf("NULL\n");
}

void liberarLista(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
}

int main() {
    No *lista = NULL;

    lista = inserirFinal(lista, 10);
    lista = inserirFinal(lista, 20);
    lista = inserirFinal(lista, 30);

    printf("Percurso direto:\n");
    exibirDireto(lista);
}

```

```

printf("Percurso reverso:\n");
exibirReverso(lista);

liberarLista(lista);

return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_16.c -o codigos_c/exemplo_16 -Wall -Wextra -std=c11 && ./
↳codigos_c/exemplo_16

```

26 Remoção em lista duplamente encadeada

Na lista duplamente encadeada, a remoção exige ajustar dois ponteiros:

anterior

próximo

Exemplo:

10 <-> 20 <-> 30

Removendo 20:

10 <-> 30

```

[ ]: %%writefile codigos_c/exemplo_17.c

#include <stdio.h>
#include <stdlib.h>
typedef struct No {
    int valor;
    struct No *anterior;
    struct No *proximo;
} No;

No* removerValor(No *inicio, int valor) {
    No *atual = inicio;

    while (atual != NULL && atual->valor != valor) {
        atual = atual->proximo;
    }

    if (atual == NULL) {
        printf("Valor %d não encontrado.\n", valor);
        return inicio;
    }
}

```

```

    if (atual->anterior != NULL) {
        atual->anterior->proximo = atual->proximo;
    } else {
        inicio = atual->proximo;
    }

    if (atual->proximo != NULL) {
        atual->proximo->anterior = atual->anterior;
    }

    free(atual);

    return inicio;
}

int main() {
    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_17.c -o codigos_c/exemplo_17 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_17

```

27 Lista duplamente encadeada circular

27.1 Conceito

A lista duplamente encadeada circular combina duas ideias:

1. Cada nó aponta para o próximo e para o anterior.
2. O último nó aponta para o primeiro.
3. O primeiro nó aponta de volta para o último.

Representação:

```

-----
|                               \
10 <-> 20 <-> 30 <-> 40
/\_____

```

De forma lógica:

```

primeiro->anterior = último
último->proximo = primeiro

```

27.2 Aplicações

- playlists circulares com opção de voltar e avançar;
- carrosséis de imagens;
- jogos em turnos;
- gerenciamento circular de processos;

- menus circulares em interfaces.

28 Inserção em lista duplamente encadeada circular

Neste exemplo, vamos inserir elementos no final da lista.

```
[9]: %%writefile codigos_c/exemplo_18.c

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *anterior;
    struct No *proximo;
} No;

No* inserirDuplaCircular(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return inicio;
    }

    novo->valor = valor;

    if (inicio == NULL) {
        novo->proximo = novo;
        novo->anterior = novo;
        return novo;
    }

    No *ultimo = inicio->anterior;

    novo->proximo = inicio;
    novo->anterior = ultimo;

    ultimo->proximo = novo;
    inicio->anterior = novo;

    return inicio;
}

void exibirDireto(No *inicio) {
    if (inicio == NULL) {
        printf("Lista vazia.\n");
    }
}
```

```

        return;
    }

    No *atual = inicio;

    do {
        printf("%d <-> ", atual->valor);
        atual = atual->proximo;
    } while (atual != inicio);

    printf("(início)\n");
}

void exibirReverso(No *inicio) {
    if (inicio == NULL) {
        printf("Lista vazia.\n");
        return;
    }

    No *ultimo = inicio->anterior;
    No *atual = ultimo;

    do {
        printf("%d <-> ", atual->valor);
        atual = atual->anterior;
    } while (atual != ultimo);

    printf("(último)\n");
}

void liberarDuplaCircular(No *inicio) {
    if (inicio == NULL) {
        return;
    }

    No *atual = inicio->proximo;

    while (atual != inicio) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }

    free(inicio);
}

int main() {

```

```

    No *lista = NULL;

    lista = inserirDuplaCircular(lista, 10);
    lista = inserirDuplaCircular(lista, 20);
    lista = inserirDuplaCircular(lista, 30);
    lista = inserirDuplaCircular(lista, 40);

    printf("Percurso direto:\n");
    exibirDireto(lista);

    printf("Percurso reverso:\n");
    exibirReverso(lista);

    liberarDuplaCircular(lista);

    return 0;
}

```

Writing codigos_c/exemplo_18.c

```

[10]: !gcc codigos_c/exemplo_18.c -o codigos_c/exemplo_18 -Wall -Wextra -std=c11 && ./
      ↪codigos_c/exemplo_18

```

Percurso direto:
 10 <-> 20 <-> 30 <-> 40 <-> (início)
 Percurso reverso:
 40 <-> 30 <-> 20 <-> 10 <-> (último)

29 Comparação sem tabela: visão geral das estruturas

29.1 Vetor

Forma de armazenamento:
 elementos em posições consecutivas da memória.

Acesso:
 direto pelo índice.

Inserção no início:
 exige deslocamento.

Remoção no início:
 exige deslocamento.

Tamanho:
 geralmente fixo.

Indicado para:

situações em que o tamanho é conhecido e o acesso rápido é importante.

29.2 Lista encadeada simples

Forma de armazenamento:
nós espalhados na memória, ligados por ponteiros.

Acesso:
sequencial, do início até o elemento desejado.

Inserção no início:
rápida.

Remoção:
feita por ajuste de ponteiros.

Tamanho:
dinâmico.

Indicado para:
situações com muitas inserções e remoções.

29.3 Lista encadeada circular

Forma de armazenamento:
semelhante à lista simples, mas o último nó aponta para o primeiro.

Acesso:
sequencial e circular.

Fim da lista:
não usa NULL como fim.

Indicado para:
processos cíclicos, rodízios, turnos e playlists.

29.4 Lista duplamente encadeada

Forma de armazenamento:
cada nó aponta para o anterior e para o próximo.

Acesso:
sequencial, mas permite ida e volta.

Inserção e remoção:
mais flexíveis, porém exigem mais cuidado com ponteiros.

Custo:
usa mais memória por nó.

Indicado para:
históricos, navegação reversa e sistemas com voltar/avançar.

29.5 Lista duplamente encadeada circular

Forma de armazenamento:
cada nó aponta para anterior e próximo, e a lista forma um ciclo.

Acesso:
circular nos dois sentidos.

Indicado para:
playlists, carrosséis, jogos e sistemas com navegação contínua.

30 Comparação por perguntas

30.1 Quando preciso acessar rapidamente uma posição específica?

Melhor escolha: vetor.
Motivo: permite acesso direto por índice.

30.2 Quando o tamanho da lista muda muito?

Melhor escolha: lista encadeada.
Motivo: cresce e diminui dinamicamente.

30.3 Quando preciso inserir e remover muitos elementos no início?

Melhor escolha: lista encadeada simples ou lista duplamente encadeada.
Motivo: evita deslocamentos como ocorre no vetor.

30.4 Quando preciso percorrer continuamente os elementos?

Melhor escolha: lista circular.
Motivo: o último elemento volta ao primeiro.

30.5 Quando preciso andar para frente e para trás?

Melhor escolha: lista duplamente encadeada.
Motivo: cada nó possui ponteiro para o anterior e para o próximo.

30.6 Quando preciso andar para frente e para trás de forma circular?

Melhor escolha: lista duplamente encadeada circular.
Motivo: combina navegação dupla com comportamento circular.

31 Comparação por complexidade

31.1 Acesso por índice

Vetor:
 $O(1)$

Listas encadeadas:
 $O(n)$

31.2 Busca por valor

Vetor:
 $O(n)$

Lista encadeada simples:
 $O(n)$

Lista circular:
 $O(n)$

Lista duplamente encadeada:
 $O(n)$

31.3 Inserção no início

Vetor:
 $O(n)$, pois exige deslocamento.

Lista encadeada simples:
 $O(1)$

Lista circular:
 $O(1)$, dependendo da implementação.

Lista duplamente encadeada:
 $O(1)$

31.4 Inserção no final

Vetor:
 $O(1)$, se houver espaço disponível.

Lista encadeada simples:
 $O(n)$, se não houver ponteiro para o último nó.

Lista duplamente encadeada:
 $O(n)$, se não houver ponteiro para o último nó.

Lista duplamente circular:

$O(1)$, se o início aponta para o último por meio de início->anterior.

31.5 Remoção no início

Vetor:

$O(n)$

Lista encadeada simples:

$O(1)$

Lista duplamente encadeada:

$O(1)$

31.6 Remoção no meio

Vetor:

$O(n)$, por causa do deslocamento.

Listas encadeadas:

$O(n)$, pois normalmente é preciso buscar o elemento antes de remover.

32 Cuidados comuns em C

Ao trabalhar com listas em C, alguns erros são muito comuns.

32.1 Esquecer de verificar se malloc retornou NULL

Problema:

o programa pode tentar usar memória que não foi alocada.

Solução:

sempre verificar se o ponteiro é NULL.

32.2 Perder o endereço do início da lista

Problema:

se o ponteiro inicial for perdido, não será possível acessar a lista.

Solução:

atualizar o início com cuidado nas funções de inserção e remoção.

32.3 Esquecer de liberar memória

Problema:

vazamento de memória.

Solução:

usar free para liberar os nós.

32.4 Criar loop infinito em lista circular

Problema:

a lista circular não termina em NULL.

Solução:

parar a repetição quando o ponteiro voltar ao início.

32.5 Ajustar apenas um ponteiro na lista dupla

Problema:

a ligação entre os nós pode ficar quebrada.

Solução:

em listas duplas, atualizar anterior e próximo corretamente.

33 Resumo visual final

33.1 Vetor

[10]	[20]	[30]	[40]
0	1	2	3

Características:

acesso rápido

memória contínua

tamanho mais rígido

remoções e inserções podem exigir deslocamento

33.2 Lista encadeada simples

10 -> 20 -> 30 -> NULL

Características:

usa ponteiros

cresce dinamicamente

anda apenas para frente

33.3 Lista circular simples

10 -> 20 -> 30
^ |
|_____|

Características:

não termina em NULL

último aponta para o primeiro

boa para ciclos

33.4 Lista duplamente encadeada

NULL <- 10 <-> 20 <-> 30 -> NULL

Características:

anda para frente
anda para trás
usa mais memória

33.5 Lista duplamente encadeada circular

```
-----  
|           \/  
10 <-> 20 <-> 30  
/\_____|
```

Características:

circular
navegação nos dois sentidos
mais flexível
mais complexa

34 Exercícios conceituais

34.1 Exercício 1

Explique com suas palavras o que é uma lista linear.

34.2 Exercício 2

Qual é a diferença entre acessar um elemento em um vetor e acessar um elemento em uma lista encadeada?

34.3 Exercício 3

Por que a inserção no meio de um vetor pode ser custosa?

34.4 Exercício 4

Por que uma lista encadeada precisa usar ponteiros?

34.5 Exercício 5

Explique a diferença entre lista encadeada simples e lista circular.

34.6 Exercício 6

Explique por que a lista duplamente encadeada permite percorrer os elementos em ordem reversa.

34.7 Exercício 7

Dê um exemplo de aplicação real para cada estrutura:

```
vetor
lista encadeada simples
lista circular
lista duplamente encadeada
lista duplamente encadeada circular
```

34.8 Exercício 8

Por que uma lista circular pode causar loop infinito se for percorrida incorretamente?

35 Exercícios práticos em C

35.1 Exercício 1

Crie um programa em C que armazene 5 números inteiros em um vetor e exiba todos os valores.

35.2 Exercício 2

Crie uma função que receba um vetor e retorne o maior valor armazenado.

35.3 Exercício 3

Implemente uma função para inserir um valor no final de uma lista usando vetor.

35.4 Exercício 4

Implemente uma função para remover um elemento de uma posição específica de um vetor.

35.5 Exercício 5

Crie uma lista encadeada simples que permita inserir valores no início.

35.6 Exercício 6

Crie uma lista encadeada simples que permita inserir valores no final.

35.7 Exercício 7

Implemente uma função que busque um valor dentro de uma lista encadeada simples.

35.8 Exercício 8

Implemente uma função que remova um valor específico de uma lista encadeada simples.

35.9 Exercício 9

Implemente uma lista circular simples com inserção no final e exibição dos elementos.

35.10 Exercício 10

Implemente uma lista duplamente encadeada com exibição direta e reversa.

35.11 Exercício 11

Implemente uma lista duplamente encadeada circular com inserção no final.

35.12 Exercício 12

Crie um menu em C com as seguintes opções:

- 1 - Inserir
- 2 - Remover
- 3 - Buscar
- 4 - Exibir
- 0 - Sair

Use uma lista encadeada simples como estrutura principal.

36 Exemplos específicos de listas lineares: pilha e fila

Além das listas lineares gerais, duas estruturas aparecem com muita frequência em algoritmos, sistemas operacionais, compiladores, navegadores, aplicativos e sistemas de atendimento:

- **Pilha;**
- **Fila.**

As duas são listas lineares porque armazenam elementos em sequência. A diferença está na regra de entrada e saída dos dados.

Lista linear geral:

permite inserir, remover e acessar elementos conforme a implementação.

Pilha:

o último que entra é o primeiro que sai.

Fila:

o primeiro que entra é o primeiro que sai.

36.1 Pilha

A **pilha** segue a regra **LIFO**:

LIFO = Last In, First Out

Último a entrar, primeiro a sair

Exemplo do dia a dia:

Pilha de pratos

Coloca prato 1

Coloca prato 2

Coloca prato 3

Ao retirar:

Sai primeiro o prato 3

Depois o prato 2

Depois o prato 1

Representação:

Topo

↓

[30]

[20]

[10]

Operações principais:

push: inserir no topo

pop: remover do topo

peek/top: consultar o topo

isEmpty: verificar se está vazia

36.2 Pilha usando vetor em C

Na pilha com vetor, usamos uma variável chamada `topo` para indicar a última posição ocupada.

Quando a pilha está vazia:

`topo = -1`

Quando inserimos um elemento:

`topo` aumenta uma posição

valor é armazenado em `pilha[topo]`

Quando removemos um elemento:

valor do `topo` é removido

`topo` diminui uma posição

```
[ ]: %%writefile codigos_c/exemplo_19.c

#include <stdio.h>
#define TAM 5

void push(int pilha[], int *topo, int valor) {
    if (*topo == TAM - 1) {
        printf("Pilha cheia. Não é possível inserir %d.\n", valor);
        return;
    }

    (*topo)++;
    pilha[*topo] = valor;
}
```

```

    printf("Valor %d inserido na pilha.\n", valor);
}

int pop(int pilha[], int *topo) {
    if (*topo == -1) {
        printf("Pilha vazia. Não é possível remover.\n");
        return -1;
    }

    int valor = pilha[*topo];
    (*topo)--;
    return valor;
}

int peek(int pilha[], int topo) {
    if (topo == -1) {
        printf("Pilha vazia.\n");
        return -1;
    }

    return pilha[topo];
}

void exibirPilha(int pilha[], int topo) {
    if (topo == -1) {
        printf("Pilha vazia.\n");
        return;
    }

    printf("Pilha do topo para a base:\n");

    for (int i = topo; i >= 0; i--) {
        printf("[%d]\n", pilha[i]);
    }
}

int main() {
    int pilha[TAM];
    int topo = -1;

    push(pilha, &topo, 10);
    push(pilha, &topo, 20);
    push(pilha, &topo, 30);

    exibirPilha(pilha, topo);

    printf("Topo atual: %d\n", peek(pilha, topo));
}

```

```

    printf("Removido: %d\n", pop(pilha, &topo));

    exibirPilha(pilha, topo);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_19.c -o codigos_c/exemplo_19 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_19

```

36.3 Análise da pilha com vetor

Características principais:

Acesso ao topo:
 $O(1)$

Inserção no topo:
 $O(1)$

Remoção do topo:
 $O(1)$

Limitação:
 o tamanho máximo precisa ser definido previamente.

A pilha com vetor é simples, rápida e eficiente quando já sabemos o tamanho máximo aproximado da estrutura.

36.4 Pilha usando lista encadeada em C

Na pilha com lista encadeada, o topo será o primeiro nó da lista.

Inserir no topo significa inserir no início da lista.

Remover do topo significa remover o primeiro nó da lista.

Representação:

Topo
 ↓
 [30] -> [20] -> [10] -> NULL

Essa implementação não depende de um tamanho fixo, pois os nós são criados dinamicamente com malloc.

```

[ ]: %%writefile codigos_c/exemplo_20.c

```

```

#include <stdio.h>
#include <stdlib.h>

```

```

typedef struct No {
    int valor;
    struct No *proximo;
} No;

No* push(No *topo, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return topo;
    }

    novo->valor = valor;
    novo->proximo = topo;

    printf("Valor %d inserido na pilha.\n", valor);

    return novo;
}

No* pop(No *topo) {
    if (topo == NULL) {
        printf("Pilha vazia. Não é possível remover.\n");
        return NULL;
    }

    No *remover = topo;
    int valor = remover->valor;

    topo = topo->proximo;

    free(remover);

    printf("Valor %d removido da pilha.\n", valor);

    return topo;
}

void exibirPilha(No *topo) {
    if (topo == NULL) {
        printf("Pilha vazia.\n");
        return;
    }

    No *atual = topo;

```

```

    printf("Topo -> ");

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

void liberarPilha(No *topo) {
    No *atual = topo;

    while (atual != NULL) {
        No *proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
}

int main() {
    No *topo = NULL;

    topo = push(topo, 10);
    topo = push(topo, 20);
    topo = push(topo, 30);

    exibirPilha(topo);

    topo = pop(topo);

    exibirPilha(topo);

    liberarPilha(topo);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_20.c -o codigos_c/exemplo_20 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_20

```

36.5 Análise da pilha com lista encadeada

Características principais:

Inserção no topo:

$O(1)$

Remoção do topo:
 $O(1)$

Consulta ao topo:
 $O(1)$

Vantagem:
não precisa definir tamanho máximo fixo.

Cuidado:
é obrigatório liberar memória com `free`.

A pilha com lista encadeada é indicada quando a quantidade de elementos pode variar bastante durante a execução do programa.

37 Fila

A fila segue a regra **FIFO**:

FIFO = First In, First Out
Primeiro a entrar, primeiro a sair

Exemplo do dia a dia:

Fila de banco

Pessoa 1 chega primeiro
Pessoa 2 chega depois
Pessoa 3 chega depois

Ao atender:
Pessoa 1 sai primeiro
Pessoa 2 sai depois
Pessoa 3 sai depois

Representação:

Início				Fim
↓				↓
[10]	->	[20]	->	[30] -> [40]

Operações principais:

enqueue: inserir no fim da fila
dequeue: remover do início da fila
front: consultar o primeiro elemento
isEmpty: verificar se está vazia

37.1 Fila usando vetor em C

Em uma fila com vetor, controlamos duas posições:

inicio: indica quem será removido primeiro
fim: indica onde o próximo valor será inserido

Nesta primeira versão, a fila avança no vetor. É uma implementação didática e simples.

Limitação desta versão:

Após muitas remoções, podem sobrar espaços vazios no começo do vetor.
Por isso, em aplicações reais, costuma-se usar fila circular.

```
[ ]: %%writefile codigos_c/exemplo_21.c

#include <stdio.h>
#define TAM 5

void enqueue(int fila[], int *fim, int valor) {
    if (*fim == TAM - 1) {
        printf("Fila cheia. Não é possível inserir %d.\n", valor);
        return;
    }

    (*fim)++;
    fila[*fim] = valor;
    printf("Valor %d inserido na fila.\n", valor);
}

int dequeue(int fila[], int *inicio, int fim) {
    if (*inicio > fim) {
        printf("Fila vazia. Não é possível remover.\n");
        return -1;
    }

    int valor = fila[*inicio];
    (*inicio)++;

    return valor;
}

void exibirFila(int fila[], int inicio, int fim) {
    if (inicio > fim) {
        printf("Fila vazia.\n");
        return;
    }

    printf("Início -> ");

    for (int i = inicio; i <= fim; i++) {
        printf("%d -> ", fila[i]);
    }
}
```

```

    printf("Fim\n");
}

int main() {
    int fila[TAM];
    int inicio = 0;
    int fim = -1;

    enqueue(fila, &fim, 10);
    enqueue(fila, &fim, 20);
    enqueue(fila, &fim, 30);

    exibirFila(fila, inicio, fim);

    printf("Removido: %d\n", dequeue(fila, &inicio, fim));

    exibirFila(fila, inicio, fim);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_21.c -o codigos_c/exemplo_21 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_21

```

37.2 Fila circular usando vetor

Para aproveitar melhor o vetor, podemos implementar uma **fila circular**.

A ideia é fazer o índice voltar para o começo quando chega ao final do vetor.

Índices do vetor:

0 1 2 3 4

Quando o fim chega ao índice 4,
o próximo avanço pode voltar para o índice 0,
se houver espaço disponível.

A movimentação circular é feita com o operador %:

```
proxima_posicao = (posicao_atual + 1) % TAM;
```

Essa técnica é muito usada em buffers, filas de impressão, comunicação entre processos e sistemas embarcados.

```

[ ]: %%writefile codigos_c/exemplo_22.c

```

```

#include <stdio.h>
#define TAM 5

```



```

typedef struct {
    int dados[TAM];
    int inicio;
    int fim;
    int quantidade;
} FilaCircular;

void inicializar(FilaCircular *fila) {
    fila->inicio = 0;
    fila->fim = -1;
    fila->quantidade = 0;
}

void enqueue(FilaCircular *fila, int valor) {
    if (fila->quantidade == TAM) {
        printf("Fila cheia. Não é possível inserir %d.\n", valor);
        return;
    }

    fila->fim = (fila->fim + 1) % TAM;
    fila->dados[fila->fim] = valor;
    fila->quantidade++;

    printf("Valor %d inserido.\n", valor);
}

int dequeue(FilaCircular *fila) {
    if (fila->quantidade == 0) {
        printf("Fila vazia. Não é possível remover.\n");
        return -1;
    }

    int valor = fila->dados[fila->inicio];
    fila->inicio = (fila->inicio + 1) % TAM;
    fila->quantidade--;

    return valor;
}

void exibirFila(FilaCircular fila) {
    if (fila.quantidade == 0) {
        printf("Fila vazia.\n");
        return;
    }

    printf("Início -> ");

```

```

    int posicao = fila.inicio;

    for (int i = 0; i < fila.quantidade; i++) {
        printf("%d -> ", fila.dados[posicao]);
        posicao = (posicao + 1) % TAM;
    }

    printf("Fim\n");
}

int main() {
    FilaCircular fila;

    inicializar(&fila);

    enqueue(&fila, 10);
    enqueue(&fila, 20);
    enqueue(&fila, 30);
    enqueue(&fila, 40);

    exibirFila(fila);

    printf("Removido: %d\n", dequeue(&fila));
    printf("Removido: %d\n", dequeue(&fila));

    enqueue(&fila, 50);
    enqueue(&fila, 60);

    exibirFila(fila);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_22.c -o codigos_c/exemplo_22 -Wall -Wextra -std=c11 && ./
    ↪codigos_c/exemplo_22

```

37.3 Análise da fila com vetor

Na fila com vetor simples:

Inserção no fim:

$O(1)$

Remoção do início:

$O(1)$, se apenas avançarmos o índice início

Problema:

pode haver desperdício de posições no começo do vetor


```

void enqueue(Fila *fila, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro ao alocar memória.\n");
        return;
    }

    novo->valor = valor;
    novo->proximo = NULL;

    if (fila->inicio == NULL) {
        fila->inicio = novo;
        fila->fim = novo;
    } else {
        fila->fim->proximo = novo;
        fila->fim = novo;
    }

    printf("Valor %d inserido na fila.\n", valor);
}

int dequeue(Fila *fila) {
    if (fila->inicio == NULL) {
        printf("Fila vazia. Não é possível remover.\n");
        return -1;
    }

    No *remover = fila->inicio;
    int valor = remover->valor;

    fila->inicio = fila->inicio->proximo;

    if (fila->inicio == NULL) {
        fila->fim = NULL;
    }

    free(remover);

    return valor;
}

void exibirFila(Fila fila) {
    if (fila.inicio == NULL) {
        printf("Fila vazia.\n");
        return;
    }
}

```

```

    No *atual = fila.inicio;

    printf("Início -> ");

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("Fim\n");
}

void liberarFila(Fila *fila) {
    while (fila->inicio != NULL) {
        dequeue(fila);
    }
}

int main() {
    Fila fila;

    inicializar(&fila);

    enqueue(&fila, 10);
    enqueue(&fila, 20);
    enqueue(&fila, 30);

    exibirFila(fila);

    printf("Removido: %d\n", dequeue(&fila));

    exibirFila(fila);

    liberarFila(&fila);

    return 0;
}

```

```

[ ]: !gcc codigos_c/exemplo_23.c -o codigos_c/exemplo_23 -Wall -Wextra -std=c11 && ./
↪codigos_c/exemplo_23

```

37.5 Análise da fila com lista encadeada

Características principais:

Inserção no final:

$O(1)$, quando existe ponteiro para o fim

Remoção do início:
 $O(1)$

Consulta ao primeiro elemento:
 $O(1)$

Vantagem:
o tamanho cresce conforme a necessidade

Cuidado:
é necessário gerenciar memória dinamicamente

A fila com lista encadeada é muito útil quando não sabemos previamente quantos elementos serão armazenados.

38 Comparação didática entre pilha e fila

38.1 Regra de funcionamento

Pilha:
último que entra é o primeiro que sai.

Fila:
primeiro que entra é o primeiro que sai.

38.2 Onde a inserção acontece

Pilha:
insere no topo.

Fila:
insere no fim.

38.3 Onde a remoção acontece

Pilha:
remove do topo.

Fila:
remove do início.

38.4 Exemplos práticos

Pilha:

- desfazer/refazer em editores;
- chamadas de funções;
- navegação entre páginas;
- análise de expressões matemáticas.

Fila:

- fila de impressão;
- fila de processos no sistema operacional;
- fila de atendimento;
- mensagens aguardando processamento.

39 Comparação entre implementações com vetor e lista encadeada

39.1 Pilha com vetor

Boa quando:

- o tamanho máximo é conhecido;
- deseja-se uma implementação simples;
- não há necessidade de crescimento dinâmico.

Limitação:

- pode ficar cheia.

39.2 Pilha com lista encadeada

Boa quando:

- o tamanho pode variar;
- deseja-se alocação dinâmica;
- não se sabe previamente a quantidade de elementos.

Limitação:

- exige uso de ponteiros, malloc e free.

39.3 Fila com vetor

Boa quando:

- o tamanho máximo é conhecido;
- usa-se fila circular para aproveitar o espaço;
- busca-se eficiência com estrutura simples.

Limitação:

- o tamanho máximo continua fixo.

39.4 Fila com lista encadeada

Boa quando:

- a quantidade de elementos varia muito;
- é necessário crescer dinamicamente;
- inserções e remoções precisam ser rápidas.

Limitação:

- exige mais cuidado com ponteiros e liberação de memória.

40 Exercícios complementares sobre pilha e fila

40.1 Exercício 1

Implemente uma pilha com vetor que armazene números inteiros e permita:

1. inserir um valor;
2. remover um valor;
3. consultar o topo;
4. exibir todos os elementos.

40.2 Exercício 2

Altere a pilha com vetor para impedir inserções quando ela estiver cheia e remoções quando ela estiver vazia.

40.3 Exercício 3

Implemente uma pilha com lista encadeada para armazenar códigos de produtos.

40.4 Exercício 4

Crie uma fila com vetor para simular uma fila de atendimento com senhas numéricas.

40.5 Exercício 5

Implemente uma fila circular com vetor e teste a reutilização das posições iniciais após algumas remoções.

40.6 Exercício 6

Implemente uma fila com lista encadeada para armazenar nomes de pessoas aguardando atendimento.

40.7 Exercício 7

Explique, com suas palavras, por que a pilha é adequada para operações de desfazer.

40.8 Exercício 8

Explique, com suas palavras, por que a fila é adequada para sistemas de atendimento.

41 Fechamento

As listas lineares são fundamentais para o estudo de Estrutura de Dados.

O vetor é simples, rápido para acesso direto e adequado quando o tamanho é conhecido.

A lista encadeada simples é mais flexível, pois permite crescimento dinâmico.

A lista circular é útil quando os elementos precisam ser percorridos continuamente.

A lista duplamente encadeada permite navegação em dois sentidos.

A lista duplamente encadeada circular combina flexibilidade, navegação dupla e comportamento cíclico.

Resumo final:

Vetor:

rápido para acessar, menos flexível para alterar.

Lista encadeada simples:

flexível, dinâmica, percorre para frente.

Lista circular:

ideal para ciclos.

Lista duplamente encadeada:

permite ir e voltar.

Lista duplamente circular:

permite ir, voltar e continuar em ciclo.